



Structure-Enhanced Graph Neural ODE Network for Temporal Link Prediction

Jinlin Hou^{1,2} , Xuan Guo¹ , Jiye Liu¹ , Jie Li¹ , Lin Pan³ ,
and Wenjun Wang^{1,2}

¹ College of Intelligence and Computing, Tianjin University, Tianjin 300350, China
{houjinlin, guoxuan, jayliu7319, jieleo}@tju.edu.cn

² Georgia Tech Shenzhen Institute, Tianjin University, Shenzhen 518055, China
wjwang@tju.edu.cn

³ School of Marine Science and Technology, Tianjin University, Tianjin 300072, China
linpan@tju.edu.cn

Abstract. Temporal link prediction aims to predict future links by learning the structural information and temporal evolution of a network. However, existing methods are heavily dependent on the latest snapshots, which hinders their power to reveal the essential evolutionary patterns based on and leverage them for dynamical inference. As a result, they generally achieve better predictions for the closest future snapshots than remote ones. Moreover, most methods do not take into account the effects of higher-order and global structure. To address these issues, we propose Structure-Enhanced Graph Neural Ordinary Differential Equation (SEGODE), a framework effectively performing dynamic inference by neural ordinary differential equation incorporating attention mechanisms and empowering to capture higher-order and global structure. To validate the proposed model, we conduct multiple experiments on a total of seven real datasets. The experimental results show that the SEGODE not only achieves good performance in link prediction but also maintains excellent results even under data-scarce conditions.

Keywords: Temporal Network · Link Prediction · Network Representation Learning · Neural Ordinary Differential Equation

1 Introduction

The real world is full of ever-changing systems, and temporal networks are used to describe these complex systems in various fields, including social networks [9], academic networks [17] and viral distribution networks [12]. Link prediction, which involves predicting future links based on the structural information already available in the network, is a crucial problem in temporal networks. Frequently, temporal networks may be portrayed as a sequence of snapshots. Investigating

J. Hou and X. Guo—contribute equally.

how to derive from these snapshots the evolutionary principles of the network structure that buttress link prediction is a worthwhile conundrum.

Temporal network representation learning (TNRL) is now mainstream for learning the evolving network structure and solving the link prediction problem due to its eminent expressive ability and efficiency. The primary objective of temporal network representation is to generate low-dimensional embeddings which preserve the major information of the temporal network. This process typically involves two stages, i.e., snapshot structure encoding and temporal evolution learning.

To encode the structure information of each snapshot, deep learning methods employing graph neural networks (GNNs) have gained widespread popularity recently. However, most of these approaches primarily focus on capturing node proximity to achieve good link prediction results, and only a few algorithms consider the impact of the higher-order and global structure (**Challenge 1**). Ignoring the role of higher-order and global structure in temporal link prediction may limit the effectiveness of the learned embeddings. The higher-order structure can provide a simple and effective means of characterizing specific rules that can enhance link prediction, such as Triadic Closure [1]. In addition, the global structure can capture the inclination of nodes to be linked in the graph based on metrics such as node degree and PageRank (PR) [13].

The route to capturing temporal information in the deep learning manner has three main branches. The first category involves recurrent neural network (RNN) approaches. Dyngraph2vec [5] proposes inputting the structural information of each snapshot obtained by the encoder into Long Short-Term Memory (LSTM) to learn the temporal information. EvolveGCN [14] combines static structure learning with a static GNN and LSTM/Gate Recurrent Unit (GRU) to encode the temporal information. The second category involves attention mechanisms. DySAT [16] uses attention mechanisms at both static and dynamic scales to learn the representation of the temporal graph, while STGSN [11] globally constrains the model by taking into account the historical topology and incorporating global information in the computation of attention weights. However, these two categories of methods are highly dependent on the latest snapshots and generally perform better for the closest future snapshots than remote ones. They lack dynamical inference uncovering the true evolutionary patterns (**Challenge 2**). Furthermore, obtaining the most immediate training data is usually not feasible in the real world. Moreover, the above methods flounder in this situation as they cannot directly predict snapshots for multiple future moments (**Challenge 3**). Real-world temporal networks may also suffer from snapshot loss due to data corruption, which we call scarce data in this paper. Similarly, the above methods also flounder in this situation as they cannot handle missing snapshots (**Challenge 4**). Recently, the third category of methods leveraging neural ordinary differential equations (ODEs) [3] has been proposed. On the one hand, the dynamic rules of most real-world systems can be elucidated by ODEs. On the other hand, by solving ODEs in a neural network framework, neural ODEs can learn opaque dynamic patterns underlying extremely complex systems like the

social network. Through dynamic inference step by step, it can directly predict multiple future moments while maintaining the ability to handle snapshot missing. NDCN [20] is the first approach that combines neural ODEs with graph neural networks to obtain low-dimensional representations of temporal graphs. While NDCN can solve Challenges 2, 3, and 4, its simple structural encoding cannot effectively learn structural features, and the fixed ODE function limits its ability to handle the temporal network with varying topologies.

To overcome the limitations mentioned above, this paper proposes a novel framework for temporal link prediction called **Structure-Enhanced Graph Neural Ordinary Differential Equation (SEGODE)**. In order to address Challenge 1, SEGODE introduces the Graphlet Degree Vector (GDV) [7] method and PR [13] for learning the snapshot features from the perspective of higher-order and global structure, respectively. To tackle Challenges 2, 3, and 4, we incorporate the neural ODE with an attention mechanism to characterize dynamical systems. The proposed method in this paper effectively refines the snapshot graph structure encoding and captures the dynamic rules of temporal evolution. In summary, our contributions are as follows:

1. This article points out that learning dynamic rules and being able to infer based on them are necessary for learning temporal network representations. Meanwhile, higher-order and global structure information are needed for better dynamic inference. However, contemporary approaches are inadequate in both aspects.
2. We propose SEGODE, a framework for temporal link prediction, that effectively captures the evolving rules of temporal patterns through the use of a neural ODE with an attention mechanism and the enhancement via multi-scale structure encoding.
3. We compare our proposed model with the baselines on real networks and conduct experiments on single-step link prediction, multi-step prediction, and data-scarce link prediction. The results demonstrate that our model not only exhibits excellent link prediction performance but also performs exceptionally well on multi-step link prediction and data-scarce link prediction.

2 Related Work

2.1 Temporal Network Representation Learning

TNRL has received significant attention recently, owing to its relevance in various domains. Some of the previous approaches are through matrix factorization. This technique exploits the correlation between the node embeddings of different snapshots to learn the dynamics of temporal graphs. They generally go through different optimization rules to reconstruct the adjacency relations and learn the dynamics of the network through snapshot smoothing. For instance, LIST [19] employs a multi-step label propagation method on each snapshot and models the dynamic topology as a function of temporal correlation. MLjFE [10] learns higher-order approximation by multi-label learning and handles network dynamics through a temporal smoothing term.

In recent years, several researchers have explored deep learning-based methods to tackle the problem of learning temporal network representation. These approaches typically leverage neural networks to learn both adjacency relationships and temporal evolution between multiple snapshots. For example, DynGEM [6] employs an auto-encoder to learn the structural information of each snapshot, and it initializes each snapshot using the parameters of the last one, which accelerates convergence while maintaining embeddings stability between snapshots. Dyngraph2vec [5] considers the impact of multiple moments on the current state, capturing highly nonlinear interactions between multiple snapshots. GNNs are commonly utilized to learn the structural information of snapshots and are combined with mechanisms learning temporal information to jointly accomplish temporal networks. EvolveGCN [14] updates the graph convolutional network (GCN) [8] parameters at each moment through RNNs, allowing the model parameters to adapt to more recent snapshots. DySAT [16] leverages the extended graph attention network and Transformer to incorporate temporal information, enabling the model to capture more salient information. In addition to learning adjacencies using GCN and attention layers in each snapshot individually, STGSN [11] improves graph structure learning by stacking global temporal adjacencies obtained from the entire historical snapshot. The latest approach ROLAND [18] extends static GNNs to achieve dynamic representation learning by fusing the hidden representations of neighboring snapshot GNNs to learn the evolutionary laws. The above approach can be summarized as the graph-then-time approach. Relatively, the time-then-graph model [4] can also be used for network representation learning. It is no less capable than the graph-then-time approach, which is proved by both theoretical analysis and a typical representative of this type of method GRU-GCN experimentally.

3 Preliminaries

3.1 Problem Definition

Temporal Network. We define a temporal network as $G = \{\mathcal{G}^0, \mathcal{G}^1, \dots, \mathcal{G}^T\}$, where $\mathcal{G}^t = (\mathcal{V}^t, \mathcal{E}^t)$ denotes a network snapshot at time t . $\mathcal{V}^t$ is a set node at time t , and we denote by $\mathcal{V} = \mathcal{V}^0 \cup \mathcal{V}^1 \cup \dots \cup \mathcal{V}^T = \{v_0, v_1, \dots, v_{N-1}\}$ the set of N nodes among all the historical snapshots. The structure of each snapshot can be represented by the binary adjacency matrix $\mathbf{A}^t \in \mathbb{R}^{N \times N}$. The attributes of all nodes in $\mathcal{V}^t$ are represented by $\mathbf{X}^t \in \mathbb{R}^{N \times F}$, where $F \ll N$ is the number of attributes. In this paper, we leverage GDV and PR separately to measure the higher-order and global structure of each snapshot. $\mathbf{L}^t$ and $\mathbf{P}^t$ denote the GDV matrix and PR matrix of the snapshot at time t , respectively.

Temporal Network Representation Learning. Given a temporal network G , the goal of TNRL is to obtain a low-dimensional representation $\mathbf{z}_i^t$ for each node v_i at time t based on the adjacencies and attributes of each snapshot. The representations of all nodes at time t are given by the matrix $\mathbf{Z}^t$. The learned representations would be used for downstream tasks.

Link Prediction. Set the current time as τ . The single-step link prediction task aims to predict the adjacency matrix $\mathbf{A}^\tau$ of the network at time τ using the historical snapshot $\{\mathcal{G}^0, \mathcal{G}^1, \dots, \mathcal{G}^{\tau-1}\}$. In this paper, we also extend the single-step task to multi-step link prediction, i.e., predicting the adjacencies of the future l snapshots from $\mathbf{A}^\tau$ to $\mathbf{A}^{\tau+l-1}$. We also define a data-scarce link prediction task, which involves simulating missing data by deleting some historical snapshots by a certain percentage, based on the single-step link prediction. The latter two tasks require a more powerful ability of the proposed model to learn the dynamics.

3.2 Neural Ordinary Differential Equations

Neural Ordinary Differential Equations can model continuous dynamics and solve Initial Value Problems (IVP) by giving initial values [3]. For a future state $z(t_1)$, it can be obtained by solving from the initial state $z(t_0)$:

$$z(t_1) = z(t_0) + \int_{t_0}^{t_1} f(z(t), t, \boldsymbol{\theta}) dt, \quad (1)$$

where $f(z(t), t, \boldsymbol{\theta})$ is a neural network used to fit the differential $\frac{dz(t)}{dt}$, which is a mapping to control changes in the dynamical system, often referred to as the ODE function. $z(t_i)$ represents the state of the system at time t_i and $t_i \in [0, T]$ refers to a time-point. $\boldsymbol{\theta}$ is the set of parameters. The entire ordinary differential equation is computed by a black-box ODE solver, with adjoint sensitivity method [15] used to compute the gradient. By providing a time series and the initial state of the system, the solution for the time series can be obtained. Due to the advantages of memory efficiency, adaptive training, and continuous time series modeling, neural ODEs are worth being introduced to TNRL.

4 The Proposed Model

4.1 Overview

In this section, we proffer SEGODE, a framework for the dynamic inference of temporal graphs. Figure 1 presents the overview of SEGODE which comprises three fundamental modules including (a) Structure Encoding, (b) Evolution Learning, and (c) Representation Decoding. Preprocessing is carried out on the temporal network, obtaining local, higher-order, and global structural information by the adjacency matrix, GDV matrix and PR matrix for each network snapshot. After inputting them into the Structure Encoding module, they are encoded separately through the three corresponding encoders to acquire the structural embedding of each snapshot. In the evolution learning module, we create the ODE function using the attention mechanism on obtained structural embeddings and then temporally update the node hidden embeddings via the ODE solver. Finally, after transmitting the hidden embeddings through the representation decoding model, we get the final node representations and train them via both snapshot reconstruction and snapshot smoothness.

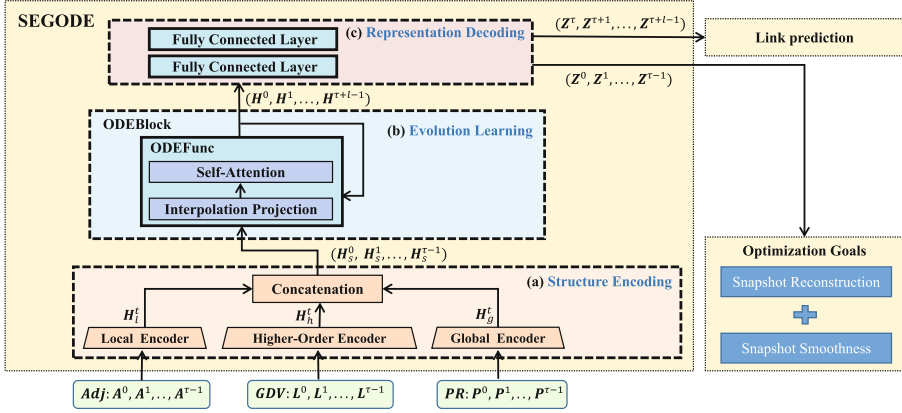


Fig. 1. An overview of the SEGODE framework. SEGODE consists of three modules: structure encoding, evolution learning, and representation decoding. Optimization is performed through two objectives: snapshot reconstruction and snapshot smoothness.

4.2 Structure Encoding

This module aims to comprehensively learn structural information from different perspectives, which is the basis of learning evolutionary dynamics. We construct three encoders to learn the local, higher-order, and global structure of each snapshot graph respectively.

To learn the local proximity among the nodes, we employ two GCN [8] layers as the local encoder f_l on A^t and X^t . Through this process, we obtain the local structural embedding H_l^t at time t :

$$H_l^t = f_l^{(1)}(A^t, f_l^{(0)}(A^t, X^t)), \quad f_l(A, X) = \text{Tanh}(\tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}} X W_l), \quad (2)$$

where $\tilde{A} = A + I$ and $\tilde{D}$ is the degree matrix of $\tilde{A}$. W_l represent the learnable parameters. For graphs without attributes, the node degree is leveraged to construct the attribute matrix.

In the higher-order and global encoder, two distinct mapping functions are employed to map GDV [7] matrix L^t and PR [13] matrix P^t into the hidden space. The mapping functions $f_h(L^t)$ and $f_g(P^t)$ are characterized by nonlinear layers, which result in higher-order embedding H_h^t and global embedding H_g^t :

$$H_h^t = f_h(L^t) = \sigma(L^t W_h + b_h), \quad H_g^t = f_g(P^t) = \sigma(P^t W_g + b_g). \quad (3)$$

σ is sigmoid activation and W_h , W_g , b_h and b_g are the learnable parameters.

Ultimately, the three aforementioned embeddings are merged by a fusion function denoted as f_{con} to acquire H_s^t :

$$H_s^t = f_{con}(H_l^t \| H_h^t \| H_g^t), \quad (4)$$

where $\|$ denotes the concatenation operation. f_{con} is defined by a multi-layer perceptron (MLP) comprised of two layers with Tanh activation function.

4.3 Evolution Learning

To use the neural ODE to compute the final embeddings of the desired snapshot, we need to transform the discrete-time embeddings $\{\mathbf{H}_s^0, \mathbf{H}_s^1, \dots, \mathbf{H}_s^{\tau-1}\}$ obtained from the structural encoding into continuous-time embeddings. Thus by interpolation projection, the continuous representation $\mathbf{H}_{ip}^t$ is obtained as:

$$\mathbf{H}_{ip}^t = \frac{\lceil t \rceil_e - t}{\lceil t \rceil_e - \lfloor t \rfloor_e} \mathbf{H}_s^{\lfloor t \rfloor_e} + \frac{t - \lfloor t \rfloor_e}{\lceil t \rceil_e - \lfloor t \rfloor_e} \mathbf{H}_s^{\lceil t \rceil_e}, \quad (5)$$

where $\lfloor t \rfloor_e$ is crafted to explore downwards up to a non-negative integer, and the embedding at time $\lfloor t \rfloor_e$ ought to be non-empty. This process can effectively address data-scarce cases. If the data is complete, $\lfloor t \rfloor_e$ means rounding down. Correspondingly, the $\lceil t \rceil_e$ explores upwards. If $t = 0$, then $\mathbf{H}_{ip}^0 = \mathbf{H}_s^0$. Moreover, when t is larger than $\tau - 1$, $\mathbf{H}_{ip}^t = \mathbf{H}_s^{\tau-1}$.

We utilize the attention mechanism to construct the ODE function:

$$f_{ode}(\mathbf{H}^t, t, \boldsymbol{\theta}) = \text{softmax}\left(\frac{\mathbf{Q}^t \mathbf{K}^{t\top}}{\sqrt{d_k}}\right) \mathbf{V}^t, \quad (6)$$

$$\mathbf{Q}^t = \mathbf{H}_{ip}^t \mathbf{W}_q, \quad \mathbf{K}^t = \mathbf{H}_{ip}^t \mathbf{W}_k, \quad \mathbf{V}^t = \mathbf{H}^t, \quad (7)$$

where lies the task of fitting the t moment hidden embedding, denoted as $\mathbf{H}^t$, through the utilization of a neural ODE. $\mathbf{H}^t$ denotes the state matrix of the system at time t . d_k is the embedding dimension. $\boldsymbol{\theta}$ includes the weight matrices $\mathbf{W}_q$ and $\mathbf{W}_k$. We obtain the input to the neural ODE in time t_0 by a single-layer MLP i.e., $f_{mlp}(\mathbf{H}_{ip}^0)$. Finally, the neural ODE can be expressed as:

$$\mathbf{H}^{t_1} = f_{mlp}(\mathbf{H}_{ip}^0) + \int_0^{t_1} f_{ode}(\mathbf{H}^t, t, \boldsymbol{\theta}) dt. \quad (8)$$

By designating $\mathcal{T}$ as a series of discrete moments $\mathcal{T} = \{0, 1, \dots, \tau + l - 1\}$ and $t_1 \in \mathcal{T}$, we are able to procure the hidden embeddings at each of these specific instances through Eq. (8). Specifically, we use Runge-Kutta of order 5 of Dormand-Prince-Shampine [2] as the ODE solver to implement Eq. (8).

4.4 Model Optimization

After obtaining $\mathbf{H}^t$ through the evolution learning module, we proceed to pass it through the representation decoding module prior to conducting model optimization. The final representation set $\{\mathbf{Z}^0, \mathbf{Z}^1, \dots, \mathbf{Z}^{\tau+l-1}\}$ used for optimization and link prediction is generated by the module via two fully connected layers. Optimization is performed on $\{\mathbf{Z}^0, \mathbf{Z}^1, \dots, \mathbf{Z}^{\tau-1}\}$. Our approach to optimizing the model is two-fold: snapshot reconstruction and snapshot smoothness.

Snapshot Reconstruction. To enable the node representation to preserve information pertaining to its adjacent nodes, we utilize the binary cross-entropy

function identical to that of DySAT [16] for every individual time step:

$$\mathcal{L}_r = \sum_{t=0}^{\tau-1} \sum_{i \in \mathcal{V}} \left(\sum_{j \in \mathcal{P}_i^t} -\log \sigma(\langle \mathbf{z}_i^t, \mathbf{z}_j^t \rangle) - \alpha \sum_{j' \in \mathcal{N}_i^t} \log(1 - \sigma(\langle \mathbf{z}_i^t, \mathbf{z}_{j'}^t \rangle)) \right), \quad (9)$$

where $\mathbf{z}_i^t = \mathbf{Z}_{i,:}^t$, and $\langle \mathbf{a}, \mathbf{b} \rangle$ denotes the inner product of $\mathbf{a}$ and $\mathbf{b}$. $\mathcal{P}_i^t$ represents the set of positively sampled neighbors of node v_i at time t_i that are obtained through random walks. The degree distribution $\mathcal{N}_i^t$ is utilized for negative sampling. The hyperparameter α serves the purpose of regulating the respective contributions of positive and negative sampling.

Table 1. Dataset statistics.

Dataset	$ \mathcal{V} $	$ \cup_t \mathcal{E}^t $	Min. $ \mathcal{E}^t $	Max. $ \mathcal{E}^t $	#Snapshots
Email	1,891	5,870	102	2,861	6
UCI	1,899	15,675	291	9,015	7
Bitcoin	3,783	14,380	945	3,113	8
Reality	6,809	16,623	607	1,351	18
Wiki	8,227	32,856	1,045	2,744	14
Reddit	10,985	292,222	9,015	11,869	27
Math	24,740	230,956	4,899	13,162	26

Snapshot Smoothness. Maintaining the smoothness of two adjacent snapshot representations is a crucial optimization objective. Using snapshot smoothness can impose penalties on mutated node representations within the network. Our objective function for snapshot smoothness using L1-norm is as follows:

$$\mathcal{L}_s = \sum_{t=1}^{\tau-1} \sum_{i \in \mathcal{V}} \|\mathbf{z}_i^t - \mathbf{z}_i^{t-1}\|_1. \quad (10)$$

By leveraging a hyperparameter β , we ultimately obtain the final objective function which facilitates the adjustment of the respective contributions of both snapshot reconstruction and snapshot smoothness:

$$\mathcal{L} = \mathcal{L}_r + \beta \mathcal{L}_s. \quad (11)$$

5 Experiments

5.1 Experimental Setup

Datasets and Baselines. The seven used real-world datasets include Email-dnc (shortly as Email), UCI, Bitcoinalpha (shortly as Bitcoin), Reality-call (shortly as Reality), Wikipedia (shortly as Wiki), Reddit, and Math. Statistical details of these datasets are given in Table 1.

We apply seven temporal graph link prediction algorithms for comparison. For the RNN-based methods, we choose Dyngraph2vec [5] (best version: DynAERNN), EvolveGCN [14] (best version: EvolveGCN-H) and ROLAND [18] (best version: ROLAND-GRU). We also use attention mechanism-based approaches DySAT [16] and STGSN [11], an ODE-based method NDCN [20], and an implementation of the time-then-graph paradigm, namely the GRU-GCN [4].

Table 2. AUC (%) on the inductive single-step link prediction.

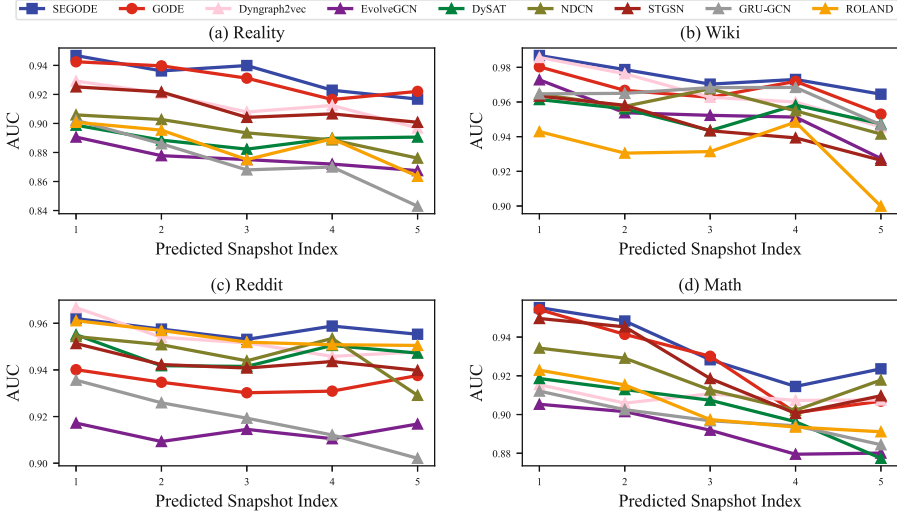
Methods	Email	UCI	Bitcoin	Reality	Wiki	Reddit	Math
Dyngraph2vec	97.6 $\pm$ 0.9	87.0 $\pm$ 1.1	89.6 $\pm$ 0.7	96.4 $\pm$ 0.4	93.4 $\pm$ 0.2	96.7 $\pm$ 0.1	91.9 $\pm$ 1.5
EvolveGCN	96.9 $\pm$ 1.2	85.9 $\pm$ 2.1	92.2 $\pm$ 0.4	92.9 $\pm$ 0.9	88.7 $\pm$ 0.8	91.9 $\pm$ 0.7	90.4 $\pm$ 0.5
DySAT	95.9 $\pm$ 1.3	87.3 $\pm$ 1.8	87.8 $\pm$ 1.7	96.5 $\pm$ 0.3	89.3 $\pm$ 0.2	96.5 $\pm$ 0.3	93.3 $\pm$ 0.9
NDCN	95.4 $\pm$ 1.2	80.4 $\pm$ 3.3	85.3 $\pm$ 1.4	96.7 $\pm$ 31.3	88.2 $\pm$ 1.5	95.3 $\pm$ 2.7	92.3 $\pm$ 0.8
STGSN	<u>97.9 $\pm$ 0.2</u>	<u>89.3 $\pm$ 2.7</u>	82.9 $\pm$ 2.5	93.0 $\pm$ 1.2	<u>94.1 $\pm$ 2.5</u>	94.3 $\pm$ 1.3	<u>95.6 $\pm$ 0.4</u>
GRU-GCN	95.0 $\pm$ 2.5	83.6 $\pm$ 1.6	91.0 $\pm$ 0.8	96.4 $\pm$ 0.6	88.7 $\pm$ 1.7	92.9 $\pm$ 1.1	91.1 $\pm$ 1.1
ROLAND	96.5 $\pm$ 1.8	88.5 $\pm$ 0.9	94.7 $\pm$ 1.6	93.1 $\pm$ 0.4	92.7 $\pm$ 1.4	95.0 $\pm$ 0.9	91.6 $\pm$ 1.3
SEGODE	98.3 $\pm$ 0.6	89.8 $\pm$ 1.0	96.6 $\pm$ 0.7	97.9 $\pm$ 0.4	94.5 $\pm$ 1.3	96.6 $\pm$ 0.3	95.7 $\pm$ 0.2

Parameter Setting and Evaluation Metric. We set the size of node representations to 256. For optimization, we use the Adam algorithm with a learning rate of 0.01, weight decay of 0.0005, and a dropout rate of 0.05. We set the random walk length to 20 in the snapshot reconstruction loss and the number of negative samples to 10 for each positive edge. The positive and negative sampling contribution hyperparameter α is set to 1. The regulation value β for Email, UCI, Bitcoin, Reality, Wiki, Reddit, and Math are 10, 10, 10, 10, 1, 0.0001, and 0.00001, respectively. We report the average results over 10 repeated times. The evaluation metric chosen is the area under the ROC curve (AUC).

More details of the datasets and implementation of our model are available at <https://github.com/Hou1/SEGODE>.

5.2 Single-Step Link Prediction

For single-step link prediction, the model is trained on the set $\{\mathcal{G}^0, \mathcal{G}^1, \dots, \mathcal{G}^{\tau-1}\}$. SEGODE and NDCN generate node embedding predictions directly for $\mathcal{G}^\tau$, while the others rely on the embeddings of $\mathcal{G}^{\tau-1}$ for prediction on $\mathcal{G}^\tau$. For evaluation, we use a downstream Logistic Regression (LR) classifier with equal numbers of connected and unconnected node pairs (non-links) randomly sampled from $\mathcal{G}^\tau$. We split the links into training, validation, and test sets in a 1:1:2 ratio. The results for ten runs expressed as mean $\pm$ standard deviation are presented in Table 2, where the best results are in bold, and the runner-ups are underlined. Our method performs the top results on all the datasets.

**Fig. 2.** AUC on the multi-step link prediction.**Table 3.** AUC (%) on the data-scarce link prediction.

Methods	Email	UCI	Bitcoin	Reality	Wiki	Reddit	Math	Avg. Drop
Dyngraph2vec	<u>97.3</u>	82.1	87.9	94.6	<u>92.2</u>	95.3	88.3	1.99 ↓
EvolveGCN	96.0	84.3	75.7	91.6	88.6	90.3	88.0	3.49 ↓
DySAT	88.2	82.3	87.3	<u>96.2</u>	85.6	95.0	91.3	2.96 ↓
NDCN	94.4	79.2	81.5	88.9	82.8	94.2	91.2	3.06 ↓
STGSN	68.1	<u>88.5</u>	75.5	87.5	87.2	91.1	<u>93.4</u>	7.97 ↓
GRU-GCN	95.1	78.2	81.4	95.5	86.5	90.9	86.5	3.51 ↓
ROLAND	96.2	86.5	<u>91.9</u>	92.9	86.8	94.3	88.1	2.20 ↓
SEGODE	97.4	88.7	94.5	97.9	92.6	96.5	95.4	0.91 ↓

5.3 Multi-step Link Prediction

Here we conduct the multi-step link prediction. To test the usefulness of our model design, we add GODE (SEGODE without higher-order and global encoder) as a baseline. We use the same inductive manner as before, training the model on $\{\mathcal{G}^0, \mathcal{G}^1, \dots, \mathcal{G}^{\tau-1}\}$. While SEGODE, GODE, and NDCN directly output node embeddings of $\{\mathcal{G}^\tau, \mathcal{G}^{\tau+1}, \dots, \mathcal{G}^{\tau+4}\}$, other methods predict links in these 5 snapshots using node embedding of $\mathcal{G}^{\tau-1}$. For each snapshot, we use the same way to implement the evaluation mentioned in Sec. 5.2. The experimental results, presented in Fig. 2, show that SEGODE consistently performs at a high level on multi-step prediction. GODE shows top or competitive results on three datasets, indicating that the implemented neural ODE using the attention mechanism, can effectively learn the dynamics. On Reddit with a larger average

node degree, GODE has a significant decrease in performance, indicating that the learned structural information via the structure encoding module is critical for dynamic inference in this situation.

5.4 Data-Scarce Link Prediction

To verify whether SEGODE can get rid of the strong dependence on history recent snapshots and really capture the temporal pattern of the network, we remove 20% of recent (rounding up) snapshots for each dataset. We directly apply SEGODE with the scarce data. For baselines, we use the weighted combination of adjacent undeleted snapshots to fill the gaps made by snapshot deletion, where we keep the common edges of adjacent snapshots and sample non-common edges in proportion to their temporal distance. Our results are shown in Table 3. There is instability in the effect of baselines on this experiment, with a substantial decrease in effect on several datasets. In contrast, SEGODE can still achieve relatively high results on all the datasets.

Table 4. AUC (%) for ablation study.

Methods	Email	UCI	Bitcoin	Wiki
SEGODE	98.3 ± 0.6	89.8 ± 1.0	96.6 ± 0.7	94.5 ± 1.3
SEGODE w/o f_h	97.3 ± 1.4	87.8 ± 3.2	95.3 ± 1.5	93.1 ± 1.8
SEGODE w/o f_g	97.5 ± 1.0	87.3 ± 2.3	95.3 ± 1.1	91.1 ± 2.3
SEGODE w/o ODE	93.8 ± 1.9	86.4 ± 1.1	90.6 ± 2.7	86.8 ± 2.9
SEGODE w/o $\mathcal{L}_s$	96.8 ± 1.4	87.8 ± 1.4	94.8 ± 1.1	87.7 ± 2.2
GODE	96.4 ± 1.9	87.3 ± 1.4	93.1 ± 0.8	91.5 ± 3.3

5.5 Ablation Study

To investigate the individual impact of the different components in SEGODE, we perform an ablation study via single-step link prediction with four variants: SEGODE w/o f_h , SEGODE w/o f_g , SEGODE w/o $\mathcal{L}_s$, GODE, SEGODE w/o ODE (where the ODE-based Evolution Learning module is replaced with RNN-based model). The results of our experiments are presented in Table 4. We observe the neural ODE component is the most crucial element for effectively learning temporal features compared with the RNN-based model. The absence of the snapshot smoothness loss $\mathcal{L}_s$ also substantially reduces the experimental effect, which reflects the loss $\mathcal{L}_s$ can facilitate model training. Furthermore, removing one of the structure encoders f_h and f_g reduces AUC values significantly, and removing both of them reduces the AUC values more, indicating all kinds of structural information are needed for enhancing the dynamic inference.

6 Conclusion

In conclusion, our proposed framework, SEGODE, effectively captures the temporal evolution pattern of graphs by using neural ODE with an attention mechanism. Additionally, SEGODE captures higher-order and global structural information to enhance evolution learning compared with classic methods. SEGODE performs well in multi-step link prediction and data-scarce link prediction tasks, indicating that the model effectively learns the dynamics of networks. Overall, our proposed framework provides a powerful tool for analyzing the temporal evolution of graphs and has significant potential for applications in various fields.

Acknowledgment. In this work, we were supported by the Shenzhen Sustainable Development Project (KCXFZ20211020172544004) and the Natural Science Foundation of Inner Mongolia Autonomous Region of China (2022LHMS06008).

References

1. Bianconi, G., Darst, R.K., Iacovacci, J., et al.: Triadic closure as a basic generating mechanism of communities in complex networks. *Phys. Rev. E* **90**, 042806 (2014)
2. Calvo, M., Montijano, J., Rande, L.: A fifth-order interpolant for the Dormand and prince Runge-Kutta method. *J. Comput. Appl. Math.* **29**(1), 91–100 (1990)
3. Chen, R.T., Rubanova, Y., Bettencourt, J., et al.: Neural ordinary differential equations. In: *Advances in Neural Information Processing Systems*, vol. 31 (2018)
4. Gao, J., Ribeiro, B.: On the equivalence between temporal and static equivariant graph representations. In: *International Conference on Machine Learning* (2022)
5. Goyal, P., Chhetri, S.R., Canedo, A.: dyngraph2vec: capturing network dynamics using dynamic graph representation learning. *Knowl.-Based Syst.* **187** (2020)
6. Goyal, P., Kamra, N., He, X., Liu, Y.: DynGEM: deep embedding method for dynamic graphs. In: *IJCAI International Workshop on Representation Learning for Graphs* (2018)
7. Hočevár, T., Demšar, J.: A combinatorial approach to graphlet counting. *Bioinformatics* **30**(4), 559–565 (2014)
8. Kipf, T.N., Welling, M.: Semi-supervised classification with graph convolutional networks. In: *International Conference on Learning Representations* (2017)
9. Kumar, S., Zhang, X., Leskovec, J.: Predicting dynamic embedding trajectory in temporal interaction networks. In: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining* (2019)
10. Ma, X., Tan, S., Xie, X., et al.: Joint multi-label learning and feature extraction for temporal link prediction. *Pattern Recognit.* **121** (2022)
11. Min, S., Gao, Z., Peng, J., et al.: STGSN—a spatial-temporal graph neural network framework for time-evolving social networks. *Knowl.-Based Syst.* (2021)
12. Murphy, C., Laurence, E., Allard, A.: Deep learning of contagion dynamics on complex networks. *Nat. Commun.* **12**(1), 4720 (2021)
13. Page, L., Brin, S., Motwani, R., Winograd, T.: The PageRank citation ranking: bringing order to the web. Technical report, Stanford InfoLab (1999)
14. Pareja, A., Domeniconi, G., Chen, J., et al.: EvolveGCN: evolving graph convolutional networks for dynamic graphs. In: *Proceedings of the Thirty-Fourth AAAI Conference on Artificial Intelligence* (2020)

15. Pontryagin, L.S.: Mathematical Theory of Optimal Processes. CRC Press, Boca Raton (1987)
16. Sankar, A., Wu, Y., Gou, L., et al.: DySAT: deep neural representation learning on dynamic graphs via self-attention networks. In: Proceedings of the 13th International Conference on Web Search and Data Mining (2020)
17. Tang, J., Zhang, J., Yao, L., et al.: ArnetMiner: extraction and mining of academic social networks. In: Proceedings of the 14th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (2008)
18. You, J., Du, T., Leskovec, J.: ROLAND: graph learning framework for dynamic graphs. In: Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (2022)
19. Yu, W., Cheng, W., Aggarwal, C.C., et al.: Link prediction with spatial and temporal consistency in dynamic networks. In: International Joint Conference on Artificial Intelligence (2017)
20. Zang, C., Wang, F.: Neural dynamics on complex networks. In: Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (2020)